# Computing Graph Edit Distance via Neural Graph Matching

**Abhinav Chowdary Bikkina - 23B1018**
**Kunsoth Chandrashen Nayak - 23B1066**
**Gnana Koushik - 23B1000**
**Lohit Adhitya - 23B0952**

## Abstract

Graph Edit Distance (GED) is a fundamental measure for quantifying structural dissimilarity between graphs, defined as the minimum number of primitive edit operations required to transform one graph into another. Despite its usefulness in chemical molecule comparison, program analysis, and graph retrieval, exact GED computation is NP-hard [4]. Recent GNN-based approaches [1, 2] approximate GED efficiently but treat it as a pure regression task, producing no edit path and potentially returning *infeasible* predictions below the true GED. The GEDGNN framework [9] addresses this by jointly predicting a GED value and a node-level matching matrix, from which a concrete, feasible edit path is extracted via $k$-best bipartite matching.

## 1 Introduction

Graphs provide a natural representation for relational data and appear widely in social networks, biology, chemistry, and program analysis. **Graph Edit Distance (GED)**—the minimum number of atomic edit operations converting $G_1$ into $G_2$—is one of the most expressive structural similarity measures, capturing both topological and attribute-level differences. The sequence of operations achieving this minimum is the *graph edit path*, which provides an interpretable account of how two graphs differ.

Exact GED is NP-hard [4], and A*-based exact methods [3] become intractable beyond ≈16 nodes. Classical approximations via A*-beam search [8] or bipartite-matching greedy algorithms [7, 10] trade optimality for speed but suffer from poor solution quality. GNN-based models [1, 2, 12] learn GED from data with low mean absolute error (MAE) at near-linear inference time, but treat GED as a *pure regression task*: no edit path is produced, and predictions can be *infeasible* (below the true GED)—what [9] terms *invisible solutions*—for which no valid edit path exists. GEDGNN [9] resolves this by learning a node-level matching matrix alongside the GED value, then using $k$-best bipartite matching to generate a concrete, feasible edit path.

# 2 Problem Formulation

Following [9], a graph is $G = (V, E, L)$ where $V$ is the vertex set, $E \subseteq V \times V$ is the edge set, and $L$ assigns a label to each node.

**Definition 1** (Graph Edit Distance [9])**.** $\mathrm{GED}(G_1, G_2)$ *is the minimum number of primitive operations converting* $G_1$ *to* $G_2$*, where the operations are: (1) adding or removing an edge; (2) adding or removing an isolated node; (3) relabelling a node.*

**Definition 2** (Graph Edit Path [9])**.** *A sequence* $(o_1, \ldots, o_k)$ *of primitive operations transforming* $G_1$ *into* $G_2$ *is an* edit path*. For any edit path,* $k \geq \mathrm{GED}(G_1, G_2)$*, so its length is a valid upper bound on the true GED.*

Every edit path induces a node correspondence between $V_1$ and $V_2$, modelled as a bipartite matching.

**Definition 3** (Maximum Bipartite Matching [9])**.** *Given* $G = (V_1, V_2, E)$ *with* $E \subseteq V_1 \times V_2$*, a matching* $M \subseteq E$ *with no shared endpoints is a* maximum bipartite matching *if* $|M| = \min(|V_1|, |V_2|)$*. A weighted variant assigns* $w : E \to \mathbb{R}$*.*

**Edit path from matching.** Given a maximum bipartite matching $M$ on $(V_1, V_2)$, an edit path is produced in $O(n+m)$ time by: (1) relabelling each matched pair $(u, v)$ if $L(u) \neq L(v)$, adding/removing unmatched nodes; (2) adding/removing edges to align adjacency [9]. We write $\mathrm{GED}(G_1, G_2, M)$ for the induced cost.

**Observation 4.** $\mathrm{GED}(G_1, G_2, M) \geq \mathrm{GED}(G_1, G_2)$ *for any* $M$*. Hence any algorithm that explicitly constructs an edit path yields a feasible upper bound, ruling out invisible solutions.*

**Problem 5** (GED with Edit Path)**.** *Given training pairs* $\{(G_1, G_2, \mathrm{GED}(G_1, G_2))\}$*, learn a model that for any unseen* $(Q_1, Q_2)$*: (a) accurately estimates* $\mathrm{GED}(Q_1, Q_2)$*; and (b) produces a concrete, feasible edit path of length at most the predicted value.*

# 3 Literature Review

## 3.1 Classical Exact and Approximate Algorithms

**Exact methods.** A* search [3] finds optimal edit paths but is exponential in the number of nodes, becoming intractable beyond ≈16 nodes. A*-beam search [8] caps the search queue via *beamsize* to find near-optimal solutions faster, but retains exponential worst-case complexity.

**Greedy bipartite matching.** Methods such as Hungarian [7] and VJ [6] reduce GED to a linear assignment over hand-crafted node-pair costs, solvable in $O(n^3)$. While tractable, approximation errors are large due to weak heuristics [9].

## 3.2 GNN-Based Regression Models

**SimGNN** [2] uses graph-level embeddings and a Neural Tensor Network (NTN) to predict a scalar similarity score. As it operates at the graph level, it cannot model node correspondences and produces no edit path.

**TaGSim** [1] predicts the count of each edit-operation type via a graph aggregation layer (GAL). Strong on labelled graphs (e.g., AIDS), it degenerates on unlabelled data and cannot generate edit paths.

**Noah** [12] equips A*-beam search with a GNN-based heuristic (GPN) trained on GED values and edit paths. Despite this, the underlying A*-beam search is exponential; on large graphs Noah can exceed multi-hour time limits [9]. All three models share the critical flaw that predictions can be infeasible, yielding invisible solutions [9].

### 3.3 GEDGNN: Neural Graph Matching

GEDGNN [9] solves the GED problem in two coupled steps.

**Step 1 — Joint prediction.** A three-layer GIN [11] produces node embeddings $H_1, H_2$. Two *cross matrix modules* compute node-to-node interactions via $c$ learned matrices $\{W_i\}$:

$$A = \left[H_1 W_1 H_2^\top,\ \ldots,\ H_1 W_c H_2^\top\right],$$

followed by an MLP. One module outputs the *matching matrix* $A_{\text{match}} \in (0,1)^{|V_1|\times|V_2|}$, trained with binary cross-entropy against the ground-truth matching. The other outputs a *cost matrix* $A_{\text{cost}}$, with predicted GED:

$$\widehat{\text{GED}} = f\big(\text{Softmax}(A_{\text{match}}) \cdot A_{\text{cost}} + \text{bias}\big),$$

where bias comes from an NTN on graph-level embeddings. The joint loss $\mathcal{L} = \mathcal{L}_{\text{match}} + \lambda\, \mathcal{L}_{\text{value}}$ allows matching and value objectives to reinforce each other.

**Step 2 — Edit path extraction.** The $k$-best bipartite matching algorithm [5] enumerates the top-$k$ maximum-weight matchings of $A_{\text{match}}$. An improved variant integrates GED lower-bound pruning during solution-space splitting, exploring more than $k$ matchings within $k$ iterations [9]. Each matching converts to an edit path in $O(n+m)$; the shortest is returned and tightens $\widehat{\text{GED}}$ as a certified upper bound.

**Results.** GEDGNN reduces MAE over the best GNN regression baseline by 4.9%–74.3% across AIDS, Linux, IMDB-small, and IMDB-large, and reduces MAE over Noah by 53.6%–88.1% [9]. The feasible rate reaches ≈99.9–100% versus strictly sub-100% for all regression-only models.

## 4 GitHub Link

CS768 Course Project

# References


[1] Jiyang Bai and Peixiang Zhao. TaGSim: Type-aware graph similarity learning and computation. *Proceedings of the VLDB Endowment*, 15(2):335–347, 2021.

[2] Yunsheng Bai, Hao Ding, Song Bian, Ting Chen, Yizhou Sun, and Wei Wang. SimGNN: A neural network approach to fast graph similarity computation. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining (WSDM)*, pages 384–392, 2019.

[3] David B. Blumenthal and Johann Gamper. On the exact computation of the graph edit distance. *Pattern Recognition Letters*, 134:46–57, 2020.

[4] Horst Bunke. On a relation between graph edit distance and maximum common subgraph. *Pattern Recognition Letters*, 18(8):689–694, 1997.

[5] Chandra R. Chegireddy and Horst W. Hamacher. Algorithms for finding $k$-best perfect matchings. *Discrete Applied Mathematics*, 18(2):155–165, 1987.

[6] Roy Jonker and Anton Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38(4):325–340, 1987.

[7] Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

[8] Michel Neuhaus, Kaspar Riesen, and Horst Bunke. Fast suboptimal algorithms for the computation of graph edit distance. In *Proceedings of the Joint IAPR International Workshops on Statistical Pattern Recognition (SPR-SSPR)*, pages 163–172, 2006.

[9] Chengzhi Piao, Tingyang Xu, Xiangguo Sun, Yu Rong, Kangfei Zhao, and Hong Cheng. Computing graph edit distance via neural graph matching. *Proceedings of the VLDB Endowment*, 16(8):1817–1829, 2023.

[10] Kaspar Riesen and Horst Bunke. Approximate graph edit distance computation by means of bipartite graph matching. *Image and Vision Computing*, 27(7):950–959, 2009.

[11] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019.

[12] Lei Yang and Lei Zou. Noah: Neural-optimized A* search algorithm for graph edit distance computation. In *Proceedings of the 37th IEEE International Conference on Data Engineering (ICDE)*, pages 576–587, 2021.